05-6-findings

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 5 supporting write-up: every defect in the three artefacts, ranked by exploitability in this environment rather than by scanner severity, with the fix and the policy that prevents recurrence.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/04-findings.md`

Section 4 — review and remediate

Ranked by **exploitability in this environment**, not by scanner severity. A CVSS 9.8 on a code path nothing calls outranks nothing. The ranking question I am actually answering is: *what does an attacker need in order to use this, and what do they get?*

Prevention is the graded half, so each finding names the policy that structurally stops it coming back, and every policy has a fixture that fails before the fix and passes after.

(a) Workload manifest — `policy/kyverno/tests/resources/bad-workload.yaml`

#	defect	attacker needs	attacker gets	fix	prevented by
A1	`Role` grants `verbs: ["*"]` on `secrets`, `pods`, `configmaps`	**only the SA token** — an SSRF, a path traversal, or a logged `/var/run/secrets/...` read. No RCE.	read *and write* every Secret in `ns/accounts` (DB credential, KYC key), plus `create pods` → schedule a pod that mounts `hostPath: /` and get node root. Namespace-to-cluster escalation with the lowest entry bar on this list.	two verbs, named resources, no `pods` create	`restrict-rbac-wildcards.yaml`
A2	`hostNetwork: true`	code exec, or nothing at all	**voids every NetworkPolicy** — those apply to the pod network, and this pod is not on it, so the whole egress control in section 5 is decorative. Reaches the node's link-local IMDS at 169.254.169.254 (node role credentials if IMDSv1 is on anywhere in the fleet), and binds node ports. Also makes per-pod SG scoping to RDS impossible, because traffic leaves with the *node's* ENI.	delete it	`disallow-host-namespaces.yaml`
A3	`hostPath: /var/run/docker.sock`	code exec in the container	node root → every co-tenant pod's SA token and mounted secrets, i.e. the other ~12 teams. **Honest caveat:** EKS ≥ 1.24 runs containerd, so on a current node this path does not exist and kubelet silently creates an empty directory — the mount is inert *today*. It is ranked third, not first, for that reason. It stays high because the fleet is not guaranteed uniform, because `hostPath` at all is a node-filesystem primitive, and because it is one character edit from `/var/lib/kubelet` or `/`.	delete both hostPath volumes	`disallow-hostpath.yaml`
A4	`runAsUser: 0`, no `allowPrivilegeEscalation: false`, no dropped capabilities, no seccomp, writable root filesystem	code exec	root inside the container: install tooling, patch the app binary, and turn a container-runtime CVE (runc CVE-2024-21626 class) into an escape. `privileged: false` is present and reads as a control, which is precisely why this gets missed in review.	non-root UID, `drop: [ALL]`, `allowPrivilegeEscalation: false`, `seccompProfile: RuntimeDefault`, `readOnlyRootFilesystem: true`	`require-restricted-securitycontext.yaml`
A5	`envFrom: secretRef` — the DB credential enters the process environment	any RCE, any crash	`/proc/self/environ`, every child process, every core dump, and most APM/crash agents, which ship environment blocks to a third-party SaaS by default. Also: an env var cannot be re-read, so **rotation requires a pod restart**, which is what turns the 5-minute emergency rotation in section 6 into a 30-minute one.	mount as a file via the Secrets Store CSI driver; app re-reads on change	`disallow-secrets-as-env.yaml`

#	defect	attacker needs	attacker gets	fix	prevented by
A6	`image: ...:latest`	registry write, or tag mutation	with `:latest` the default `imagePullPolicy` is `Always`, so a mutated tag reaches all three replicas on the next restart **with no deploy and no Git change**. Also makes signature verification meaningless — you cannot verify a moving target.	digest reference	`require-image-digest.yaml`, `verify-image-signature.yaml`
A7	`hostPath: /var/log` mounted at `/hostlogs`	code exec	reads `/var/log/pods/**` — the stdout of every other team's containers on that node. In a bank that is a cross-tenant data incident by itself, before any privilege escalation. Mounted read-write it is also a symlink primitive against the kubelet log rotation.	delete	`disallow-hostpath.yaml`
A8	no `resources.limits`, no `topologySpreadConstraints`, no probes	code exec, or a bug	on a cluster shared with twelve teams, an unbounded container is a noisy-neighbour outage for services this team does not own. Availability, not confidentiality — hence last.	limits set in the fixed manifest	`require-resource-limits` (audit mode; see §5 rollout)

Fixed manifest: `remediate/workload-fixed.yaml`.

Prevention beyond the listed defects. Fixing A1 by narrowing the Role publishes its own bypass: a tenant who can no longer write `verbs: ["*"]` can bind their ServiceAccount to the `cluster-admin` ClusterRole that Kubernetes ships, with no wildcard anywhere in their manifest. So `restrict-privileged-rolebindings.yaml` ships *with* the wildcard policy, not after it. The same reasoning produced the `initContainers` rule — see `docs/05-admission-rollout.md`, "two blind spots".

(b) Terraform — `policy/fixtures/terraform/bad/main.tf`

#	defect	attacker needs	attacker gets	fix	prevented by
B1	`publicly_accessible = true` **and** `cidr_blocks = ["0.0.0.0/0"]` on 5432	an internet connection and a port scanner	a direct TCP path to the production database holding name, email, phone, IBAN and last-4. The only remaining control is a password (see B3). No application exploit, no cluster access, no insider. Internet-wide scanners find an open 5432 within hours of exposure. **Nothing else on this page is close.**	`publicly_accessible = false`, ingress restricted to the EKS node security group	`deny_rds_pub` `deny_open_ir`
B2	`password = var.db_password`	read access to Terraform state	the master password in **plaintext in the state file**. `aws_db_instance.password` is stored unencrypted in state by design — so the credential's blast radius is everyone with `s3:GetObject` on the state bucket and everyone who has ever run `terraform plan` locally. It is also static and never rotated.	`manage_master_user_password = true` → RDS-managed secret in Secrets Manager with native rotation; `iam_database_authentication_enabled = true` for the app path	`deny_rds_inl`
B3	`storage_encrypted = false`	AWS-layer access, or a copied snapshot	not the most *exploitable* item, but the most *expensive*: encryption cannot be enabled in place. Fixing it means snapshot → restore encrypted → cut over, i.e. a maintenance window that gets deferred, so the cost of this defect compounds daily. Snapshots exported or shared cross-account are plaintext. Direct PCI DSS 3.5 / RBI finding.	`storage_encrypted = true` with a customer-managed KMS key	`deny_rds_une`
B4	`backup_retention_period = 0`, `skip_final_snapshot = true`, no `deletion_protection`	any principal with `rds:DeleteDBInstance` — which, per (c), is *this pod*	**irrecoverable destruction.** No PITR window, no final snapshot, no deletion protection. A destructive attacker (or one bad `terraform destroy`) ends the bank's account records. Also a records-retention breach independent of any attacker.	35-day retention, `deletion_protection = true`, final snapshot required	`deny_rds_no_`
B5	CloudTrail `is_multi_region_trail = false`, `include_global_service_events = false`, `enable_log_file_validation = false`	nothing — this is the *absence* of a control	ranked last on exploitability and first on consequence. `include_global_service_events = false` means **IAM, STS, CloudFront and Route 53 events are not recorded at all** — precisely the API calls of an identity attack (`CreateAccessKey`,	all three true, org trail, dedicated log-archive account, KMS + Object Lock on the bucket	`deny_cloudtr`

#	defect	attacker needs	attacker gets	fix	prevented by
			`AttachRolePolicy`, `AssumeRole`). `is_multi_region_trail = false` means an attacker who works in `us-east-1` is invisible. `enable_log_file_validation = false` means no digest chain, so the log is not tamper-evident and is weak as regulatory evidence. Every "what evidence would you have?" answer in `DECISIONS.md` §3 depends on fixing this.		

Also fixed but not separately ranked: no `iam_database_authentication_enabled`, no `performance_insights` KMS key, no `monitoring_interval`, no `ca_cert_identifier` pin, no `auto_minor_version_upgrade`, trail bucket with no bucket policy, no Object Lock, no access logging.

Fixed configuration: **`terraform/`** (`rds.tf`, `security_groups.tf`, `cloudtrail.tf`, `kms.tf`, `ecr.tf`).

The rules had to survive the second version of the same mistake. A rule that tests `publicly_accessible == true` passes `publicly_accessible = var.public`, because the HCL parser hands the policy the string `"${var.public}"`. Likewise `cidr_blocks = var.allowed` arrives as a *string*, so a rule that iterates the list never executes; `dynamic "ingress"` hides the rules entirely; `::/0` is the IPv6 spelling of `0.0.0.0/0`; and `aws_rds_cluster` is the same database under a different resource type. Each of those is a way this exact defect ships again with a green pipeline, so the rule set **fails closed on values it cannot statically prove safe** and the shapes are committed as `policy/fixtures/terraform/evasion/`. One more trap worth naming because it is invisible in review: `body.backup_retention_period < 7` is *undefined* in Rego when the attribute is absent, so the rule silently passes a resource that never set it — the omission case, which is the one that actually happens. Every comparison now goes through `object.get(body, field, default)`.

(c) IAM policy on the pod identity role

What the original actually says. `"Effect": "Allow"` with `"NotAction"` is an allow of *every action in AWS* except the two listed. Attached to a pod. It grants, among roughly 15,000 other actions:

- `iam:CreateAccessKey`, `iam:AttachUserPolicy`, `iam:CreateRole`, `iam:PassRole` → persistent account-level administrator, from inside a container
- `cloudtrail:StopLogging`, `cloudtrail:DeleteTrail` → destroy the evidence of having done it
- `rds:DeleteDBInstance`, `rds:ModifyDBInstance` → destruction, or making the DB public (B1) without touching Terraform
- `secretsmanager:GetSecretValue` on `*`, `kms:Decrypt` on `*` → every other team's secrets in the account
- `s3:*`, `ec2:*`, `sts:AssumeRole` → lateral movement across the whole organisation

The second statement is redundant — it grants a strict subset of the first. Its presence is the tell: someone added a scoped statement and left the `NotAction` blanket above it, which is how these policies survive review. **`NotAction` with `Effect: Allow` should be treated as a syntax error in a policy repository.**

The rewritten policy is **`iam/pod-identity-policy.json`**.

One line on what the rewrite denies that the original allowed: everything except reading one named secret's `AWSCURRENT` version through the Secrets Manager VPC endpoint and reading/writing two named Kafka topics — in particular it now denies all IAM and STS (so no persistence and no role chaining), all CloudTrail mutation (so the evidence survives), every AWS region except `ap-south-1` (so exfiltration into an attacker's region fails and RBI data localisation holds), and any use of the credential from outside our VPC endpoints (so a stolen token is inert on the attacker's own machine).